Cosmos vial inspection technical report

Model adaptation and DGX Spark pilot results

Prepared for the VyFlare hackathon team
Report date 29 October 2026 | Resource snapshot 1:26:56 PM CDT

We adapted the Cosmos3-Nano visual reasoner using a small LoRA adapter and human-reviewed descriptions of vial color sequences. The experiment improved strict label agreement on nine development clips, and the saved adapter is available through a private-network browser tester. The evidence does not yet establish accuracy on an independent field-test session.

Main measured result

Measure	Pretrained reasoner	Pilot adapter
Exact color sequence	0/9 (0.0%)	4/9 (44.4%)
Exact final color label	3/9 (33.3%)	5/9 (55.6%)
Valid structured color output	7/9 (77.8%)	9/9 (100.0%)

Both partitions came from the organizer's video 5. Entire recipe groups were held out, but lighting, camera, and recording-session characteristics remain shared. These are development-set agreement scores, not production inspection accuracy. [E1, E2]

Completed work

• Created a separate fine-tuning branch and retained the original application branch.

• Audited the organizer dataset, workbook, field photos, and new recordings; imported 30 human color descriptions.

• Extracted and verified the existing Cosmos reasoner weights; trained 7,667,712 adapter parameters while freezing the base and vision weights.

• Completed a GPU smoke test and 20-update pilot, preserved checkpoints, and compared base and adapted predictions.

• Built a browser tester with base-versus-adapter comparison, result downloads, and optional human feedback.

Scope of the model

The current target is visible liquid color and its sequence over a clip. It does not verify chemical identity, measured composition, contamination, or exact delivered volume. The phone-photo mode is exploratory and has already shown an output-schema failure. [E3]

Data and supervision

We used real observed appearance as the training target and kept recipe intent separate. This avoids teaching the model to repeat an expected recipe when a recorded vial looks different.

Source	Available evidence	Role in this pilot
Organizer dataset	15 synchronized camera videos and 4,647 sampled images across five runs	Audited for context; only labeled video-5 vial clips entered this training run.
Separated video 5	30 clips; 10 recipes with three vials each	21 training clips and nine validation clips.
Human verification workbook	30 whole-clip color descriptions and existing model outputs	Human color descriptions supplied targets. Model outputs and pass/error were excluded.
Field experiments on October 28	30 high-resolution photos; six videos totaling about 27 minutes; seven sample metadata entries	Inspected but excluded from supervised training because mappings and observed-outcome labels are incomplete.
Digital twin	Discussed synthetic geometry, fills, camera views, and lighting	No synthetic data used. Current twin does not reproduce mixing or cross-contamination.

Targets and partitions

A human description such as blue to dark green became **{"observed_color_sequence": ["blue", "dark green"]}**. The video question asks about the vial under the nozzle, its changes, and shade descriptions. Recipe IDs and prior model answers are not included in the prompt or target.

All vials from recipes 6, 9, and 13 were assigned to validation: vials 4-6, 13-15, and 25-27. The remaining 21 were used for training. The pilot retains its single-session identity; the separate production data builder still requires independent session groups. Media hashes and recipe-group separation are checked. [E1, E4]

Label findings and decisions

The team confirmed T200 is red, correcting the organizer's orange description. This changes tank-reference context; it does not overwrite human-observed orange mixtures. The workbook pass formula uses at least one dispense-color match, while its note describes two. That column evaluates a separate rule and was excluded from color targets.

The original workbook showed 12/30 exact description matches and 3/17 matches for human descriptions containing transitions. Those audit counts motivated sequence training, but differ in prompt and population from the nine-clip controlled comparison. Whole-clip descriptions do not label exact transition times. [E4]

Model changes and rationale

The source was nvidia/Cosmos3-Nano snapshot e59a53c25979a090fa8706c9acc0c254a6e89b92. We extracted its Qwen3-VL-shaped reasoner into a Transformers-compatible directory. The source omni checkpoint remains unchanged; its diffusion and other generation pathways are not loaded by this inspection service. [E1]

Change	What was done	Reason
Reasoner extraction	Mapped and shape-checked all 750 expected state tensors, including 351 visual-tower tensors.	Use the existing Cosmos understanding weights without loading the full generation model.
LoRA adaptation	Added rank-8 adapters to language-attention q_proj, k_proj, v_proj, and o_proj.	Reduce trainable state and memory on one Spark; preserve a reversible base model.
Frozen base and vision	Only 7,667,712 language adapter parameters were trainable.	Limit compute and avoid full-model updates with a small dataset.
Structured response	Prompted and trained an ordered observed_color_sequence JSON list.	Make comparisons, application parsing, and reviewer corrections consistent.
Controlled video input	Used 32 uniform samples with original frame timestamps and one fixed pixel budget.	Keep baseline and adapted inputs comparable and memory bounded.
Separate artifacts	Saved adapter checkpoints and provenance separately from the base.	Enable rollback, comparison, and future experiments without replacing source weights.

What did not change

We did not retrain the full Cosmos model, update its original weight files, expand its vocabulary, train its vision encoder, or add a fluid-mixing simulator. We did not train chemical or contamination labels. JSON is a requested response contract, not an intrinsic requirement of Cosmos.

The extracted reasoner stores 17,534,247,392 bytes of weights, approximately 16.33 GiB, separately from the original checkpoint. The final adapter payload is 30,713,512 bytes, approximately 29.3 MiB. Its 288 saved adapter tensors were checked for finite values and expected parameter count. [E1, E5]

Training configuration and verification

Setting	Implemented value
Training path	Custom single-GPU Transformers and PEFT runner; not the initial four-GPU native recipe
Base precision and attention	BF16 base weights; SDPA attention
Adapters	Rank 8; alpha 16; dropout 0.05; FP32 saved adapter tensors
Optimizer	AdamW; learning rate 0.00005; weight decay 0.01
Batch and update budget	One clip per microbatch; accumulation 4; 20 optimizer updates
Memory controls	Gradient checkpointing; frozen base and vision; gradient clipping at norm 1.0
Video processing	32 uniform frames; 2,621,440 total video pixels; approximately 320 by 224 per frame
Supervision	Assistant-answer-only cross-entropy; prompt and visual-prefix tokens masked
Generation and seed	Greedy decoding; maximum 96 new tokens; seed 42
Software	PyTorch 2.10.0+cu130; Transformers 4.57.6; PEFT 0.18.1; Python 3.13.16

The 20-update pilot made 80 training clip presentations, with repeats across the 21 unique training clips. The separate smoke test made one additional presentation. It did not add an independent training sample. [E1, E2]

Compatibility issues resolved

The first extraction attempt stopped because Diffusers attention names differed from the reasoner names. We mapped those names and required complete tensor-name and shape coverage rather than substituting missing model weights.

A preprocessing check caught inconsistent prompt-prefix tokens caused by reused video metadata. Independent metadata for each processor call restored consistent timestamps and correct answer masking. We also corrected the pixel budget after confirming that the installed processor treats it as a total-video budget, not a per-frame limit.

The installed PyTorch build warns that GB10 compute capability exceeds its stated range. Actual CUDA generation, backpropagation, and optimizer updates succeeded. No installed packages were upgraded. Thirteen data/scoring tests and six serving API tests passed; saved-adapter inference was also verified in a separate process. [E2, E3]

DGX Spark resource usage

One DGX Spark and its single NVIDIA GB10 GPU were used. The host reports approximately 121 GiB unified system memory. Other Python/GPU workloads were already present and were left running. The following measurements describe our recorded jobs, not exclusive device allocation or the entire project's GPU consumption.

Measured activity	Smoke test	Main pilot
Recorded elapsed job time	136.17 s (2.27 min)	487.17 s (8.12 min)
Optimizer updates	1	20
Accumulation per update	1 clip	4 clips
Time inside optimizer-step loops	4.09 s	268.67 s (4.48 min)
Peak PyTorch allocated GPU memory	19.44 GiB	19.54 GiB
Evaluation clips per model	1	9

Combined recorded job wall time: 623.34 seconds, or 10 minutes 23 seconds (0.173 hours), including loading and before/after evaluation. Combined optimizer-loop time was 272.76 seconds, or 4 minutes 33 seconds. These elapsed totals exclude CPU extraction, preparation, troubleshooting, transfers, and ongoing tester hosting. [E1, E2]

Storage and continuing service

Observation	Recorded amount
Extracted reasoner directory on Spark	Approximately 17 GiB on disk
Main pilot run with four adapter checkpoints	Approximately 178 MiB
Pilot media and manifests	Approximately 33 MiB
Tester process elapsed time at resource snapshot	19 minutes 42 seconds; service remained running
Whole-host memory at that snapshot	52 GiB used; 69 GiB available

Storage figures cover the named directories and exclude the original model cache and other project directories. The browser tester keeps one reasoner plus its adapter resident; base comparison temporarily disables the adapter rather than loading a second full model. Hosting continues after training until the service is stopped. [E6]

What was not measured

We did not log integrated GPU utilization, power over time, energy, exclusive GPU-hours, or billable cost. Peak framework allocation is not total device/system memory, and elapsed job time is not an energy measurement. Existing workloads prevent attributing whole-device utilization to this experiment. A future run should record start/stop times and periodic power/utilization telemetry. [E6]

Evaluation and remaining failures

The base and adapter were evaluated on the same nine clips with the same prompt, sampling, and output budget. A full sequence match requires equal ordered lists after case and whitespace normalization. Final-label agreement compares the last list entry. An invalid or truncated JSON answer counts as an error; code fences are removed before parsing. [E2]

Vial	Human sequence	Adapter sequence
04	green	green
05	orange	yellow > orange
06	orange	orange
13	blue > dark green	green
14	blue > dark green	green
15	blue > dark green	green
25	orange	orange
26	orange > brown	orange
27	orange	orange

What improved

Exact sequence agreement rose from 0/9 to 4/9, and final-label agreement from 3/9 to 5/9. All nine adapted responses were valid color-list JSON, compared with seven base responses. The adapter reduced repeated output and improved some label agreement; this is more than a training-loss observation, but it remains a small development comparison.

What remains unresolved

All three blue-to-dark-green validation sequences collapsed to green. The orange-to-brown transition in vial 26 was missed, and vial 05 gained a yellow stage. Reduced resolution, sparse temporal sampling, scene clutter, and limited training coverage are plausible contributors; the experiment does not isolate their individual causes.

The main training loss changed from 1.385 at update 1 to 0.363 at update 20 on different sampled minibatches. That change is not an independent accuracy metric. Coarse human descriptions may omit short initial stages that the model reports, so some strict mismatches may also reflect label granularity. [E2]

Interpretation: no independent session was evaluated, there are only nine validation clips, and class/recipe coverage is limited. No production accuracy, contamination-detection performance, or reliable photo-classification claim follows from these scores.

Testing interface and data strategy

Browser testing now available

A Flask and Waitress service loads the saved pilot adapter and permits video upload, base-versus-adapter comparison, raw/parsed output inspection, and JSON downloads. It processes one inference request at a time. A busy request returns a retry message. Access is through the Spark's private network or VPN; no public tunnel was created. The existing application backend was not replaced. [E3]

The tester is at http://172.30.102.122:7865/. Videos are the recommended input. It samples 32 frames even from longer uploads and does not yet crop or track the target automatically. Uploads are deleted after inference; explicit feedback submissions save predictions, media identity, and corrections for later review, not automatic retraining.

A real held-out clip reproduced the pilot result after reloading the saved adapter: base clear > green, adapter green. Processing took 8.80 seconds in that smoke request, not a throughput benchmark. The exploratory photo smoke returned a scalar color instead of the requested list and was correctly marked invalid. [E3]

How the field data can help next

The high-resolution photos can support appearance review and image examples once each is linked to a physical vial, recipe, recording, and inspection stage. The photo timestamps were later than the recorded station sessions, so filename order alone is insufficient. One field recording also contains a late recipe entry; entry time must not be assumed to be fill start.

For new collection, use visible unique vial IDs, synchronized event logs, empty/filling/finished/settled observations, and fixed camera settings. Keep verified observations separate from intended recipes. If volume is a target, add actual mass/volume measurements and camera/vial calibration. Reserve whole independent sessions before training.

Role of the digital twin

No twin data was used in this run. Its current unmixed fluid layers are unsuitable as real mixing or carryover ground truth. Proposed uses are vial/liquid localization, fill geometry, matched camera views, and lighting/position variation. A homogeneous finished-liquid material could approximate verified appearance, but would still require real-data checks. Real-only and real-plus-synthetic models should be compared on the same untouched real test sessions.

Recipe history and synchronized tank/fill/cap views are also proposed extensions. Following a physical vial across stations requires identity and transport timing; simultaneous timestamps alone do not supply that correspondence. Confirmed contamination labels require supporting process evidence.

Reproducibility and future work

Repeating fine tuning

The training infrastructure and extracted base can be reused. Version the corrected or expanded labels, choose a new output directory, and train a fresh adapter from the unchanged base. Preserve prior adapters for comparison and rollback. The current pilot runner starts fresh; explicit adapter/optimizer resume is not implemented.

The largest remaining effort is data alignment and verification. Prioritize new station-camera sessions, clearer transition labels, and testing on the eventual live view. Use review assistance to select disagreements and rare conditions, while retaining a carefully reviewed test set. Do not automatically promote reviewer feedback or model-generated labels to ground truth.

Evidence and implementation registry

ID	Evidence retained in the project
E1	outputs/spark-pilot-v1/run.json; local data/local/spark-pilot-v1/experiment.json; base provenance records
E2	outputs/spark-pilot-v1/base.json, adapted.json, training.jsonl, smoke-run.json; reports/spark-pilot-v1.md
E3	reports/model-tester-guide.md; tester-video-smoke.json and tester-photo-smoke.json under outputs/spark-pilot-v1/
E4	vial-verification.xlsx; organizer recipe CSV and README; imported data/local/video5/annotations.jsonl; field CSV/JSON metadata
E5	outputs/spark-pilot-v1/artifact-verification.json; adapter-step-020/adapter_model.safetensors and adapter_config.json
E6	Read-only Spark inspection at 2026-10-29 18:26:56 UTC: ps, free, and du; snapshot reproduced in this report

Core implementation files are scripts/prepare_spark_pilot.py, extract_cosmos_reasoner.py, train_spark_lora.py, and serve_spark_adapter.py. Raw media, model weights, and local output directories are excluded from Git. The branch is codex/cosmos3-nano-finetune. Training/report commit c002959, tester commit b4c20f0, and photo-limitation commit ac289f8 identify the completed implementation. [E1-E5]

Adapter SHA-256
9dae56b981de7e7de200e39580e57cbcba81
d3efdcf940f2e025e80a41a3a5f4

Trainer SHA-256
34911780c2de0d487704a31e53817dc08
e71793b970c60e42b129cbd8eedce24

Background references: NVIDIA Cosmos3-Nano model card; NVIDIA Cosmos Framework; Hugging Face PEFT documentation. Experiment measurements come from the retained records above, not vendor performance claims.